

Project 1: Windows Volatile Acquisition

Robert Simpson

University of Maryland Global Campus

DFCS 625: Windows Forensics and Security

Randy Grizzelli

May 26, 2026

Project 1: Windows Volatile Acquisition

Report

The Windows Operating System (OS) architecture contains many main components. They are typically categorized as either user-mode or kernel-mode components. Kernel-mode components can be the Windows kernel itself, the hardware abstraction layer (HAL), the Windows executive, and the graphics system (Yosifovich et al., 2017). User-mode components can include user, system, service, and subsystem processes (Yosifovich et al., 2017). These systems work to run the basic life support and housekeeping functions that allow the operating system to run smoothly and with as little issues as possible.

The user-mode and kernel-mode elements are both critical to the operating system. As can be derived from their names, kernel-mode elements work at a deeper level, and user-mode components interact with applications. The Windows subsystem handles much of the basic life support and housekeeping functions of the operating system. Importantly, it utilizes four unique dynamic linked libraries (DLLs) that perform these functions (Yosifovich et al., 2017). These housekeeping functions range from handling the shutting down of applications to graphical driver support. The Windows Executive is similar in that it handles many of the essential managers for the OS. It handles input and output, security policies for the local machine, the process and thread manager, and much more (Solomon, 1998).

Understanding the user-mode and kernel-mode elements is helpful for digital forensics. For instance, the user-mode process, "CreateProcessWithLogonW", has the ability to logon and create a process with the user's credentials and token. Importantly, this process does not require special privileges (Yosivich et al., 2017). This user-mode process works through a series of steps

that eventually connects it to the kernel-mode process of “NTCreateUserProcess”. These processes, as well as almost all processes running in Windows, are tracked in the EPROCESS structure. This stores all the critical information such as process IDs and privileges. DFIR analysts use this to hunt for potential hidden processes and malware. Critically, this is harvested as part of volatile memory and so is time sensitive. Many crucial artifacts can be found in the EPROCESS structure, and so should be a key part of the investigation.

There are two common file systems to be aware of when working on Windows machines. First, the file allocation table (FAT) is an older and more simplistic file system than the new technology file system (NTFS). The system works as it sounds. It utilizes a master reference table, or the file allocation table, along with a backup copy (Microsoft, 2021). FAT uses a series of clusters to store data with each size of the cluster depending on the size of the volume. It is noted that FAT is best for storing 200mb of data or less (Microsoft, 2021). This is because FAT lacks a sophisticated level of organization. Rather than having a directory level organization, it stores data in a first come first serve basis (Microsoft, 2021). With all this being said, it may seem that FAT has limited uses and is not ideal for storing large amounts of data. That is true. However, forms of FAT like FAT32 and exFAT are commonly used for removable media such as USB thumb drives. The simplistic nature of FAT works great for modern removable media solutions. Forensic analysts should be aware of this as they may often come across having to image thumb drives.

The second type of file system mentioned is new technology file system or NTFS. As the name implies, this is a more recently developed file system that is much more robust. Not only does it utilize directory level organization, making it better suited for large amounts of data, it is a recoverable file system which makes it more reliable (Gerend, 2021). NTFS utilizes a

journaling or logging feature that makes it a recoverable file system. The system records all transactions as well as has checkpoints so that in the event of a system crash, it can use the transaction log to remap the data and recover the lost information (Microsoft, 2021). Another feature that separates NTFS from FAT is the master file table (MFT). The MFT acts as the master log for the file system. It keeps track of every file, their size, permissions, and timestamps (Ashcraft, 2024). The MFT is critical to digital forensics because when a file is deleted, it is not automatically removed from disk space. Instead, NTFS marks the space as free in the MFT so that it can eventually be overwritten. Depending on when a file was deleted and what was written after, the analyst may still be able to recover the supposedly deleted file. Unlike FAT, this makes it the standard file system for OS drives or any other additional drives that the machine may use. Additional features that make it ideal are the ability to set file and folder permissions using Access Control Lists and having drive encryption using BitLocker (Gerend, 2021). The last two parts are critical for forensic analysts. When arriving first on the scene, it may be noted that the drives are encrypted, leading to a possible warrant or needed ability to bypass the encryption. This feature is great for user privacy but also makes it more difficult for law enforcement.

Dynamic Link Libraries are a critical part of allowing software to run, which also makes them an attractive target for bad actors. MITRE describes DLLs simply as libraries that have some form of code that can be shared by multiple programs (MITRE, 2026). Because these are so frequently used and often not noticed by the average user, bad actors attempt to manipulate them in a variety of ways. Bad actors can perform a few tasks to manipulate DLLs or how the machine treats them to replace good ones with bad ones. DLL search order hijacking takes advantage of how the machines loads DLLs. Windows uses a search order to load DLLs in a correct order. A bad actor could place a trojan DLL and then manipulate the search order so that

it gets a higher priority than a good one (MITRE, 2026). The machine will think it is calling the correct DLL when in reality it has just loaded a malicious one. Another form of DLL hijacking is phantom hijacking. A program may call a DLL that does not exist. In this instance, the bad actor places a malicious DLL that matches the name of the non-existent one so that it gets loaded instead (MITRE, 2026). In this case, it is most likely that the program was malicious in nature to begin with, or the bad actor took advantage of poor code that left a call to a DLL that was later removed before publishing the software.

To find the DLLs, the forensic analyst should look through the processes that are running on the machine (SANS, 2015). The analyst can achieve this by using the sysinternals tool process explorer which is currently on version 17.12 according to Mark Russinovich at Microsoft (2026). It is common for bad actors to name their DLLs the same as a known good one, so the analyst must compare file sizes and where the DLL is stored. Additionally, the analyst should look to see if the same DLL appears in multiple locations or in a location that is abnormal to a known good DLL (SANS, 2015). Additionally, a deeper connection can be made between DLLs, registry keys, and digital forensics. When executables are run, they are tracked in the Application Compatibility Cache or ShimCache. The ShimCache's primary function is to ensure compatibility between applications and the operating system (Ciligot, 2024). It stores a log nested in the HKEY_LOCAL_MACHINE registry hive of the machine. This log populates the metadata of executables once they run. The metadata includes files name, file path, modification date and time, and the item ID. The ShimCache does not store information about the DLLs directly. However, when analysts compare the ShimCache data with the Application Activity Cache, or AmCache, more clues can be brought together. The AmCache, similar to ShimCache, also stores metadata about executables. More importantly, this metadata includes the SHA1 file

hash as well as information about DLLs that were loaded (Ciligot, 2024). These two logs can help the analyst create an extremely thorough timeline of events and connect the executables and DLLs that run on the machine.

Computers need to be able to manage their memory usage and allocation. Page files do this by reserving and committing address space which becomes known as virtual memory (Singh et al., 2018). This not only frees up physical memory space to be used, but it also aids in system crash dumps. To the forensic analyst, these page files can be gold mines. One thing every forensic analyst knows is the order of volatility. At the top of the order, meaning the most volatile, are CPU registers and caches. This data is lost when the machine loses power. Unlike persistent storage, which can retain memory when powered off for a limited but longer period of time, volatile memory is mostly lost when power is turned off. After CPU registers and caches is random access memory or RAM. The page file mentioned earlier acts as a copy of what the RAM had loaded at the time of use. This can include websites the user visited, documents or other files the user had open or modified, or processes that were running that could point to malware. If any of these activities had associated timestamps that could be temporarily held in the page file, this information can be used by the analyst to build out a more accurate timeline of events.

The Windows registry is one of the most important parts of the operating system. In these sets of keys and sub-keys lie the configurations for everything on the computer. From how file extensions are read to complete user configurations, this critical system runs in the background to maintain the operating system. The HKEY_CURRENT_USER registry hive stores the information for the current user, including files they have recently accessed (Singh et al., 2018). When a forensic analyst is working through an image of a computer, they can use RegistryRipper

to dissect and analyze the critical pieces of the registry (Shaadabn et al., 2016). An additional area of consideration is how malware can take advantage of the registry. If a malware is able to manipulate the registry, it could ensure persistence, so it never gets fully deleted if not removed from the registry. This is one crucial place for DFIR analysts to inspect after an incident. If the threat actor designed the malware with the goal of executing on startup, as many do, the analyst should investigate the HKEY_LOCAL_MACHINE and HKEY_CURRENT_USER and then go to \Software\Microsoft\Windows\CurrentVersion and dig in from there to see what suspicious software is there (Rothman, 2017). This will give the analysts a better picture of what was happening on the machine along with all the previous areas discussed earlier.

Unit 1 Lab

The first part of the examination began by verifying the hashes of the tools that will be used during the investigation. To ensure the integrity of the tools being used, MD5 hashes of all the tools on the H: drive were taken prior to receiving the Windows machine and then again once the analyst took possession of the Windows machine. The Compare-Object PowerShell utility was then used to compare the two text files containing the hashes of the tools. This utility only outputs the differences. As seen in Figure 1, the result of running the command showed no differences as the command did not output anything. By performing this function, the investigation can begin knowing that the tools have not been tampered with and the hashes match the original system state. This follows the Golden Rule of digital forensics which is to always take hashes before and after.

Figure 1: Results of running CompareFiles.ps1 showing the hashes match

```

Administrator: Windows PowerShell (2)

Directory: H:\

Mode                LastWriteTime         Length Name
----                -
d-----         4/28/2024    2:34 PM             Forensic_Tools
d-----         6/6/2024    3:32 PM             Lab4_RAM
d-----         6/5/2024    6:42 PM             Lab5_Registry
d-----         6/5/2024    6:44 PM             Lab6_ImageFiles
-a-----         5/7/2026    1:01 PM      87090 Compare_Tools_Hash.txt
-a-----         5/7/2026    1:11 PM      43544 Original_Tools_Hash.txt

PS H:\> .\CompareFiles.ps1
SelectString : The term 'SelectString' is not recognized as the name of a cmdlet, function, script file, or operable
program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At H:\CompareFiles.ps1:1 char:80
+ ... t (Get-Content -Path .\Original_Tools_Hash.txt | SelectString 'Hash : ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (SelectString:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS H:\> .\CompareFiles.ps1
Compare-Object : Cannot bind argument to parameter 'ReferenceObject' because it is null.
At H:\CompareFiles.ps1:1 char:33
+ ... rrenceObject (Get-Content -Path .\Original_Tools_Hash.txt | Select-Str ...
+ ~~~~~
+ CategoryInfo          : InvalidData: (:) [Compare-Object], ParameterBindingValidationException
+ FullyQualifiedErrorId : ParameterArgumentValidationErrorNullNotAllowed,Microsoft.PowerShell.Commands.CompareObje
ctCommand

PS H:\> .\CompareFiles.ps1
PS H:\> dir

Directory: H:\

Mode                LastWriteTime         Length Name
----                -
d-----         4/28/2024    2:34 PM             Forensic_Tools
d-----         6/6/2024    3:32 PM             Lab4_RAM
d-----         6/5/2024    6:42 PM             Lab5_Registry
d-----         6/5/2024    6:44 PM             Lab6_ImageFiles
-a-----         5/7/2026    1:16 PM              201 CompareFiles.ps1
-a-----         5/7/2026    1:01 PM      87090 Compare_Tools_Hash.txt
-a-----         5/7/2026    1:11 PM      43544 Original_Tools_Hash.txt

PS H:\>

```

There are some considerations to be made when imagining a Windows machine in

the field. For example, if the machine is a bare metal physical machine and not a virtual machine running on a server or type 2 hypervisor. First, before the analyst begins anything, they must understand the broad importance of the preservation, collection, and examination steps (Mohammed, 2023). These steps exist to ensure that first, the crime scene or incident scene is preserved and in its original state. The analyst should speak to the first responders to find out if anyone touched the machine. The analyst may begin taking photos of the machine and its surroundings, especially if the monitor is on and the computer logged in. Collecting these photos may be important for a criminal investigation. In terms of hardware, the analyst should utilize a known good clean external drive to store the image as well as a hardware write blocker. The purpose of the write blocker is to act as a precaution against accidentally altering the evidence. Along with this image, a hash should be taken of the image as is the Golden Rule for digital forensics. All digital evidence should be stored in special tamper proof containers to prevent remote alteration (Mohammed, 2023). A copy of the original image may be made with the original stored in evidence. This ensures a clean original image is always kept should anything go wrong in the examination phase. With the previous steps mentioned, the overall approach and concepts do not change much for the analyst. The same caution and careful execution is needed no matter the medium of the evidence.

Unit 2 Lab

The first part of lab 2 focused on memory acquisition. As the analyst on scene, it was critical to ensure a proper image of the volatile memory of the suspect's machine was taken. This was performed using FTK Imager. A folder named "Captures" was created under "Documents" to store the memory capture. The memory dump file was named

“Windowsmemdump.mem” and place in the “Captures” folder. Once this initial set up was completed, the memory collection process began. Figure 2 displays the file and folder name as well as the “Windowsmemdump.mem” file being approximately 11Gb in size.

After the volatile memory was collected, the registry hives were collected. Again, FTK Imager was used to perform this action using the “Obtain protected Files” function. The file output was again saved to the “Captures” folder as seen in Figure 2. Lastly, to ensure the integrity of the data and the investigation, the hashes of the “Captures” folder contents were taken. Again, the hashing function in FTK Imager was used as seen in Figure 3. It is critical that the hashes be taken and documented to maintain integrity. These hashes were taken on May 10, 2026. They were taken before any analysis was conducted so that it could be documented what the untouched data looked like. After the analysis is complete, hashing the data again is important so the hashes can be compared. If they match, it shows the analyst did not tamper with the data.

Figure 2: Contents of “Captures” folder containing memory dump and registry hive files

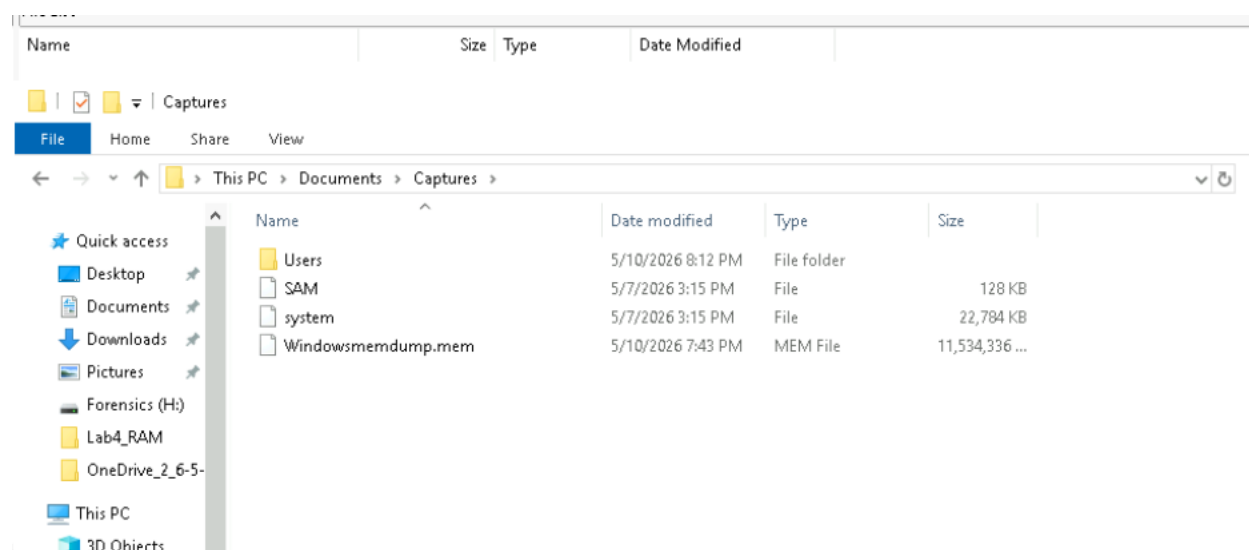


Figure 3: Hash values of the contents of “Captures” folder for the investigation

References

- Ashcraft, A. (2024, September 5). *Master File Table (Local File Systems)*. Learn.microsoft.com.
<https://learn.microsoft.com/en-us/windows/win32/fileio/master-file-table>
- Ciligot, C. (2024, October 25). *ShimCache vs AmCache: Key Windows Forensic Artifacts*. Magnet Forensics.
<https://www.magnetforensics.com/blog/shimcache-vs-amcache-key-windows-forensic-artifacts/>
- Gerend, J. (2021). *NTFS overview*. Learn.microsoft.com.
<https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-overview>
- Russinovich, M. (2024, May 28). *Process explorer - windows sysinternals*. Learn.microsoft.com.
<https://learn.microsoft.com/en-us/sysinternals/downloads/process-explorer>
- Microsoft. (2021, September 23). *Overview of FAT, HPFS, and NTFS file systems*. Learn.microsoft.com.
<https://learn.microsoft.com/en-us/troubleshoot/windows-client/backup-and-storage/fat-hpfs-and-ntfs-file-systems>
- MITRE. (2026, May 12). *Hijack execution flow: DLL search order hijacking, sub-technique T1574.001 - enterprise | MITRE ATT&CK®*. Attack.mitre.org.
<https://attack.mitre.org/techniques/T1574/001/>
- Mohammed, M. (2023, October 27). *Windows forensics analyst field guide: Engage in proactive cyber defense using digital forensics techniques*. Packt Publishing.
<https://learning.oreilly.com/library/view/windows-forensics-analyst/9781803248479/>

Rothman, A. (2017). *Windows Registry malware attacks: Knowledge is the best defense*. Red Canary.

<https://redcanary.com/blog/threat-detection/windows-registry-attacks-threat-detection/>

SANS. (2015, March 25). *Detecting DLL hijacking on Windows*. SANS Institute.

<https://www.sans.org/blog/detecting-dll-hijacking-on-window>

Shaaban, A., & Sapronov, K. (2016, June 29). *Practical Windows forensics*. Packt Publishing.

<https://learning.oreilly.com/library/view/practical-windows-forensics/9781783554096/>

Singh, A., Venter, H. S., & Ikuesan, A. R. (2018). Windows registry harnesser for incident response and digital forensic analysis. *Australian Journal of Forensic Sciences*, 52(3),

337–353. <https://www.tandfonline.com/doi/full/10.1080/00450618.2018.1551421>

Solomon, D.A. “The Windows NT Kernel Architecture.” *Computer*, vol. 31, no. 10, 1998, pp.

40–47, <https://doi.org/10.1109/2.722284>.

Yosifovich, P., Ionescu, A., Russinovich, M. E., & Solomon, D. A. (2017, May 5). *Windows internals: System architecture, processes, threads, memory management, and more*, Part 1 (7th ed.). Microsoft

Press. <https://learning.oreilly.com/library/view/windows->

[internals-part/9780133986471/](https://learning.oreilly.com/library/view/windows-internals-part/9780133986471/)